

TicTacBench: Benchmarking Timing Closure Capabilities of Coding Agents

Bowei Wang*, Zhigang Fang*, Zhijie Yang[†], Renzhi Chen[‡], Shanshan Li^{*§}, and Lei Wang^{†§}

*Computer Department, National University of Defense Technology, Changsha, China

{wangbowei, fangzhigang, shanshanli}@nudt.edu.cn

[†]Defense Innovation Institute, Academy of Military Science, Beijing, China

yangzhijie.nudt@foxmail.com, leiwang@nudt.edu.cn

[‡]Intelligent Microelectronics Center, Qiyuan Lab, Beijing, China

chenrenzhi@qiyuanlab.com

Abstract—Recent advances in large language models (LLMs) have led to the emergence of coding agents capable of performing complex engineering tasks, including register-transfer level (RTL) design and optimization. Existing RTL benchmarks mainly evaluate functional correctness and performance, power, and area (PPA) of the generated RTL designs, leaving agents’ ability for *timing closure* under-evaluated. We propose TicTacBench, a benchmark specifically designed to evaluate coding agents’ capabilities for RTL-level timing closure under post-place-and-route (post-PnR) evaluation. TicTacBench contains 30 diverse tasks, each provided with a suboptimal RTL design, realistic timing constraints, functional equivalence verification, and timing reports. With over 300 runs of coding agents driven by 8 frontier LLMs, we find that even the best agent can only close 53.3% of tasks with 7.18% area-delay product (ADP) degradation and 8.83% energy-delay-squared product (EDDP) improvement on average. We identify common failure categories that explain why agents fail to close timing. Then we propose TicTacSkill, a new method that guides agents to follow standard timing-closure procedures and improves the Timing Closure Rate by 9%. These results suggest that while coding agents have made significant progress in RTL design, their timing-closure capability still has substantial room for improvement. Anonymized code and data are available at <https://anonymous.4open.science/r/tictacbench-anon-F3C0>.

Index Terms—Electronic Design Automation, Register-Transfer Level Design, Large Language Models, Timing Closure, Evaluation Framework.

I. INTRODUCTION

Recently, coding agents driven by large language models (LLMs) have shown practical utility in complex engineering workflows [1]. This progress has led to growing interest in applying them to full hardware implementation workflows. Industrial EDA tools have adopted agent-based optimization, such as Synopsys DSO.ai and Cadence Cerebrus AI Studio. Researchers have explored LLM-based agents for RTL generation, debugging, and RTL optimization [2]–[5].

Prior studies [6]–[10] have developed several benchmarks to evaluate functional correctness and PPA (Performance, Power, Area) of agent-generated/optimized RTL. For example, VerilogEvalV2 [6] provides specification prompts, checks the functional correctness of the generated RTL through simulation, and reports $\text{func-pass}@k$ as the evaluation metric.

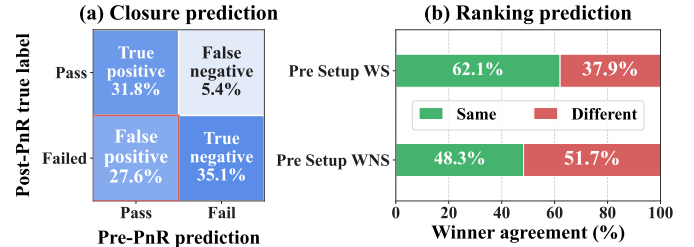


Fig. 1. Pre-PnR timing metrics are unreliable for post-PnR timing closure. (a) Treating pre-PnR closure as a prediction of post-PnR closure produces high false positives: designs that appear to close timing before PnR often fail after physical implementation. (b) Pre-PnR metrics are unreliable for evaluation. The agent selected as best before PnR disagrees with the post-PnR.

Further, RTLLM 2.0 [7] requires generated RTL to pass logic synthesis and extracts reports, using Worst Negative Slack (WNS), cells, and power as additional PPA metrics.

However, agents’ capability in *timing closure* remains underexplored. Timing closure refers to the process of optimizing a hardware design to reduce critical-path negative slack and eliminate timing violations. A design that does not close timing remains unusable even if it is functionally correct and has competitive PPA. Existing PPA benchmarks report several pre-PnR timing metrics such as WNS and Total Negative Slack (TNS). However, determining whether a design closes timing requires post-place-and-route Static Timing Analysis (post-PnR STA). As Figure 1 shows, through the statistics from Section IV, we find that using only pre-PnR WNS to judge post-PnR timing closure leads to a high false-positive rate and incorrect result rankings. This mismatch prevents us from gaining insight into agent timing-closure capability from existing benchmark results.

Research questions. Given this gap, we expect a dedicated evaluation to help answer the following research questions. **RQ1:** How capable are coding agents driven by frontier LLMs at timing-closure tasks when evaluated with post-PnR STA? **RQ2:** What failure categories explain why agents fail to close timing, indicating capability gaps? **RQ3:** How can we improve agents’ timing-closure capability beyond the baseline setting?

To address these questions, we propose **TicTacBench** (Tim-

[§]Corresponding authors.

TABLE I
COMPARISON OF REPRESENTATIVE RTL BENCHMARKS FROM THE PERSPECTIVE OF AGENT-BASED TIMING CLOSURE.

Benchmark	Task setting	Input / output	Validation artifacts	Timing-closure coverage
VerilogEval [11]	RTL generation	Prompt and target RTL	Simulation	No SDC or post-PnR objective
RTL2LLM v2 [7]	RTL generation + QoR	Spec, tests, and RTL	Cycle-accurate	QoR stress signal; no closure target
RTLBench [10]	RTL generation quality	Spec, tests, and RTL	Simulation, lint, LLM judge	No timing or PPA objective
RTL-Repo [8]	Project-scale RTL generation	Repo context and Verilog	Code checks	No task-level SDC or timing repair
RTLRewriter [4]	RTL rewriting for early QoR	RTL context and rewrite	Functional equivalence	Logic-synthesis QoR only
RTL-OPT [12]	RTL QoR optimization	Suboptimal and optimized RTL	Equivalence	PPA/QoR benchmark
CVDP [9]	Realistic RTL improvement	Project request and RTL update	Task-specific checks	Broad front-end tasks
TicTacBench (ours)	Post-PnR timing closure	Suboptimal design, SDC, reports	Interface, function, latency	Post-PnR STA with timing metrics

ing Closure through Agentic Coding Benchmark), a benchmark to evaluate coding agent capabilities for RTL-level timing closure under post-PnR evaluation. TicTacBench contains 30 diverse and challenging tasks, each provided with a suboptimal RTL design, realistic timing constraints, functional equivalence verification, pre- and post-PnR STA reports, and explicit task descriptions. During evaluation, agents can inspect the STA reports and run the provided tool set, then submit an optimized RTL design. To answer these research questions, we run more than 300 experiments with coding agents driven by 8 frontier LLMs on TicTacBench, collect agent trajectories, and evaluate every optimized design with post-PnR STA. We report four metrics that capture both timing success and physical cost: Timing Closure Rate, WNS improvement, EDDP (energy-delay-squared product) improvement, and ADP (area-delay product) improvement. We also conduct trajectory semantic analysis on the agent trajectories to identify agent capabilities and failure categories. The results show that even the GPT-5.4-based agent can close only 53.3% of tasks. The optimized designs improve EDDP by 8.83% on average, but degrade ADP by 7.18% on average. The trajectory semantic analysis shows that failure categories vary across agents, indicating gaps in RTL domain knowledge, RTL architecture choices, and optimization workflow capabilities.

Based on these findings, we propose **TicTacSkill**, which guides agents to use more suitable timing-closure procedures and improves the Timing Closure Rate by 9%. Together, these results show that coding agents still face a capability gap in industrial hardware design tasks, and point to report-grounded RTL reasoning, structural timing optimization, and flow-aware validation as promising directions for future research.

Our main contributions are as follows:

- We propose TicTacBench, to our knowledge the first benchmark that makes post-PnR timing closure the primary evaluation target for coding agents. It provides a realistic and fair evaluation for measuring whether coding agents can achieve real timing closure, a capability largely absent from existing RTL benchmarks.
- We propose several timing-closure metrics that measure both timing success and physical cost. We evaluate coding agents driven by 8 frontier LLMs over more than 300 runs and show that even the best agent closes only half of tasks, with timing gains often accompanied by area degradation.
- We analyze agent trajectories and identify 6 recurring

failure categories grouped into RTL domain knowledge, RTL architecture design, and optimization flow, indicating specific capability gaps.

- We propose TicTacSkill as a baseline method for timing closure tasks, exploring how agent capabilities can be improved for post-PnR timing closure through guidance.

II. BENCHMARKING AGENT-BASED TIMING CLOSURE

In this section, we first review representative RTL benchmarks, summarize their task settings and metrics, and analyze why no single existing benchmark fully covers the design requirements needed to evaluate agent-based timing-closure capability. Then, we revisit the definition and characteristics of timing closure in real implementation flow, and formalize the concept of Agent-based Timing Closure (ATC) Task.

A. Inspection of Current Benchmarks

In this section, we review several representative benchmarks that are widely recognized, and summarize their task settings and evaluation metrics in Table I. From the perspective of the characteristics of ATC, we argue that *no single existing benchmark covers the design requirements needed to evaluate ATC capability*. This motivates us to introduce the TicTacBench.

Lack of Timing Target/Constraints. Existing PPA benchmarks do not expose timing as a reproducible, design-specific timing-closure target. Across RTL2LLM v2, RTL-OPT, CVDP, and RTLRewriter, the reported QoR results rely on logic-synthesis artifacts that are not released as part of the task artifacts. RTL2LLM used an aggressively high frequency of 0.1 ns so that the synthesized designs always produced WNS, turning timing into a stress signal rather than a meaningful closure objective.

Cycle-Accurate Verification Bias. Existing benchmarks often rely on cycle-accurate testbenches that are stricter than timing closure requires. For example, the width_8to16 task in RTL2LLM checked `valid_out == 0` after one #10 step and later required `(data_out == 16'b1010000010100001 && valid_out == 1)` at another fixed checkpoint. As a result, cycle-accurate equivalence unnecessarily narrows the optimization space available to the agent, making latency-changing optimizations infeasible.

Lack of Post-PnR STA Ground Truth. Most existing RTL benchmarks report timing outcomes only through synthesis-stage QoR metrics such as WNS, TNS, or other PPA indica-

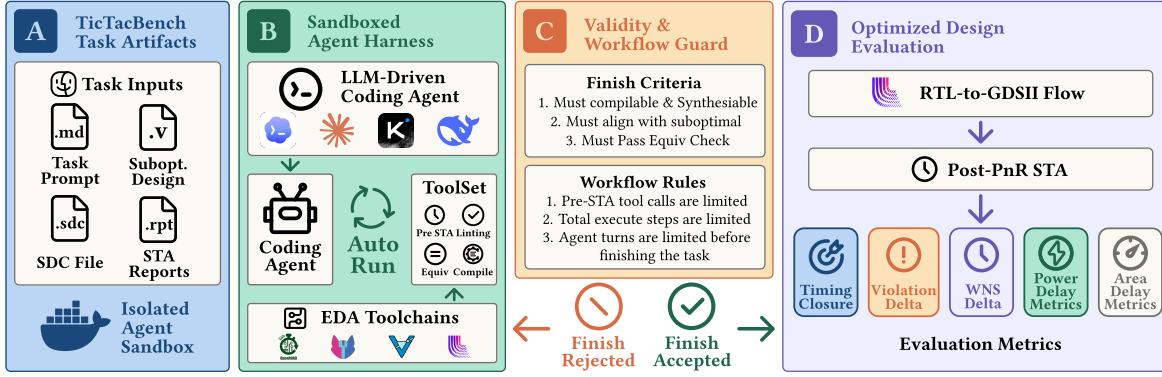


Fig. 2. TicTacBench evaluation flow. The agent uses released task artifacts and tool set commands to submit `optimized.v`; valid candidates pass interface, function, and latency guards before post-PnR evaluation computes closure, WNS repair, power-delay trade-offs, and area-delay trade-offs.

tors. These metrics cannot evaluate whether an agent can use post-PnR timing feedback to drive further RTL-level repair.

B. Definition and Characteristics of Timing Closure

Timing closure refers to the process of refining a hardware design to ensure the timing constraints of sequential elements, such as flip-flops, are satisfied under a target implementation environment [13]. Failing to achieve timing closure can lead to incorrect sampling or unstable behavior at the target frequency. The two classes of timing constraints that must be satisfied are as follows:

Setup constraint t_{setup} : requires the data at the input to remain stable at least t_{setup} before the active clock edge.

Hold constraint t_{hold} : requires the data at the input to remain stable for at least t_{hold} after the active clock edge.

Timing closure can be achieved through a variety of refinement methods, including RTL microarchitectural adjustments [14], logic synthesis optimization [15], clock-tree construction and placement-and-routing refinement [16], and post-PnR engineering change order (ECO) adjustments [17]. Microarchitectural adjustments and logic synthesis optimization are typically used to satisfy setup constraints, whereas hold repair often requires adjustments during the physical-design stage. In this paper, we focus on enabling coding agents to resolve setup violations through RTL microarchitectural optimization, thereby achieving post-PnR setup timing closure. Hold violations usually require physical-level adjustments, and the available interventions at the RTL level are much more limited; we therefore leave them out of scope.

We formalize an *ATC (Agent-based Timing Closure) task* as $x = (r, e, c, v, q, m)$, where r denotes the input RTL design; e denotes the implementation environment, including technology library, flow configuration, and timing-analysis corners that affect timing analysis and optimization results; c denotes the timing constraints, represented by SDC (Synopsys Design Constraints) files, which specify the target clock and timing assumptions under which r initially has setup violations in e ; v denotes the equivalence verification artifacts; q denotes the released timing evidence available to the agent, including baseline pre- and post-PnR reports; and m denotes any other

task-specific requirements. Let $U = \{u_{comp}, u_{equiv}, u_{sta}\}$ be the agent’s minimal tool set, corresponding to compilation, equivalence verification, and pre-PnR STA, respectively. The agent can be modeled as $A_U(x) = r'$, where the output is a new RTL design r' . The goal of the task is, for each specific input instance, to start from a design r with violations and produce an r' that either achieves timing closure or moves measurably closer to it under the same e and c .

III. BENCHMARK CONSTRUCTION AND EVALUATION

In this section, we describe how TicTacBench is constructed and how coding agents are evaluated. We then describe the evaluation flow and the post-PnR metrics used to measure timing closure, WNS repair, and the physical cost of each optimized design.

A. Task Construction

We curated candidate designs from RTLLM 2.0 [7] and RTL-OPT [12] as they provide high-quality baseline RTL with optimization headroom. We kept 25 out of 90 designs whose critical paths come from arithmetic, pipeline, or control logic. CDC and memory/storage-centered tasks are left to future benchmark splits; small standalone modules are excluded when they offer too little timing-optimization headroom.

There are two kinds of ATC tasks in TicTacBench: *latency-preserving* tasks require the optimized design to have the same latency as the baseline, while *latency-changeable* tasks allow the optimized design to have explicitly required additional latency. We derived 5 latency-changeable tasks from the original tasks, encompassing total 30 tasks in the benchmark.

B. Evaluation Flow

Figure 2 illustrates the agent harness and evaluation flow. The agent receives the task input and may inspect files, edit RTL, and run tools. The baseline post-PnR reports describe the original failure, so the agent can localize the critical path and reason from reports. During exploration, the agent runs in a sandboxed environment with the given minimal tool set. To keep runs resource-bounded and comparable, it may run the pre-PnR STA tool only a limited number of times, but

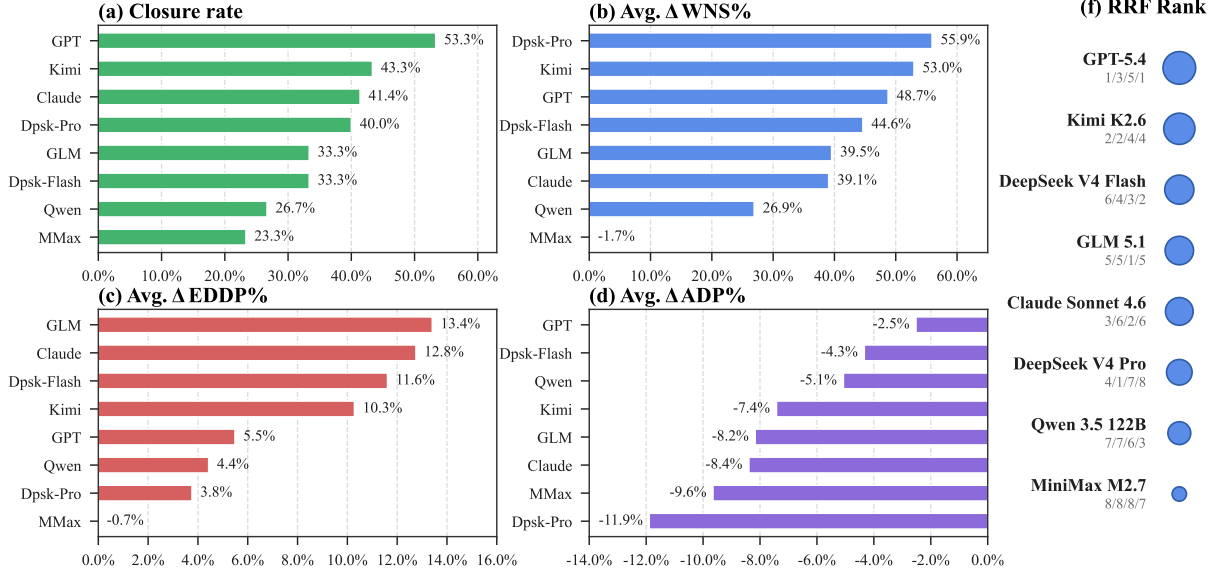


Fig. 3. Main TicTacBench leaderboard from 30 tasks and coding agents driven by 8 frontier LLMs. Panel (a) ranks agents by post-PnR Timing Closure Rate. Panels (b)–(d) rank agents by Avg. Δ WNS%, Avg. Δ EDDP%, and Avg. Δ ADP% relative to the suboptimal baseline; higher is better. Panel (e) reports the Reciprocal Rank Fusion (RRF) ranking across the four metrics.

cannot run post-PnR STA since full physical implementation has substantial resource and time cost and is usually strictly limited and scheduled in real implementation flows. The typical agent workflow is: diagnose the baseline violation, implement a synthesizable RTL transformation, check compile and equivalence, use and interpret pre-PnR STA, and submit the best final candidate.

We integrate full open source EDA toolchains for reproducibility, including Yosys for synthesis, SymbiYosys for equivalence checking, LibreLane [18] with OpenROAD and OpenSTA for full-flow post-PnR evaluation. For tasks without a latency allowance, the optimized design must be equivalent to the baseline under the same output timing. For tasks with an explicit allowed additional latency, the equivalence checker aligns the allowed cycle shift and rejects any other latency changes. After these checks pass, we rerun the full implementation flow on the optimized RTL in a clean environment.

C. Evaluation Metrics

We propose to use 4 dedicated metrics to characterize timing closure results. We first define the effective clock period t_{ECP} . For task i , let t^i be the clock period specified by the constraint, and let t_{WNS}^i be the post-PnR WNS, where closed designs have $t_{WNS}^i = 0$. The effective clock period t_{ECP} is calculated as $t_{ECP}^i = t^i - t_{WNS}^i$. This calculation intentionally ignores the positive part of slack and therefore measures relative to the constraint rather than as unconstrained timing optimization. In the following metrics, *sub.* denotes the suboptimal baseline design, and *opt.* denotes the optimized design from the agent.

The *Timing Closure Rate* r is

$$r = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[t_{WNS}^{opt,i} = 0 \right] \times 100\%, \quad (1)$$

which measures the overall fraction of benchmark tasks whose final optimized RTL is judged timing closed.

The *Average WNS improvement rate* is

$$\text{Avg. } \Delta t_{WNS}\% = \frac{1}{N} \sum_{i=1}^N \frac{t_{WNS}^{opt,i} - t_{WNS}^{sub,i}}{0 - t_{WNS}^{sub,i}} \times 100\%, \quad (2)$$

which measures the average fraction of the baseline WNS improved by the optimized design.

The *Average EDDP Improvement Rate* is

$$\begin{aligned} \text{Avg. } \Delta \text{EDDP}\% &= \frac{1}{N} \sum_{i=1}^N \frac{\text{EDDP}_i^{sub} - \text{EDDP}_i^{opt}}{\text{EDDP}_i^{sub}} \times 100\%, \\ \text{s.t. } \text{EDDP}_i &= E_{avg,i} \times (t_{ECP}^i)^2, \end{aligned} \quad (3)$$

which measures the average reduction in energy-delay-squared product, an energy-performance metric in power-aware computing [19]. We use this metric to characterize the *timing-prioritized power-timing tradeoff*.

The *Average ADP Improvement Rate* is

$$\begin{aligned} \text{Avg. } \Delta \text{ADP}\% &= \frac{1}{N} \sum_{i=1}^N \frac{\text{ADP}_i^{sub} - \text{ADP}_i^{opt}}{\text{ADP}_i^{sub}} \times 100\%, \\ \text{s.t. } \text{ADP}_i &= A_i \times t_{ECP}^i, \end{aligned} \quad (4)$$

which measures the average reduction in area-delay product, an area-timing metric in logic synthesis [20]. We use this metric to characterize the *area-timing tradeoff*.

TABLE II

OPTIMIZED POST-PNR PPA METRICS BY TASK AND MODEL. EACH MODEL GROUP REPORTS WNS, CELL COUNT, AND POWER. GREEN GROUPS PARETO-DOMINATE THE CORRESPONDING PRE-OPTIMIZATION RESULT ACROSS THE THREE METRICS, WHILE RED GROUPS ARE PARETO-DOMINATED BY IT. WNS IS IN NS, POWER IS IN MW, AND CELLS ARE COMPACTED TO CONTROL DENSITY. THE LAST ROW REPORTS DOMINATE/DOMINATED COUNTS OVER ALL TASKS.

Task	GPT			Claude			Dpsk-Pro			Dpsk-Flash			Kimi			MiniMax			GLM			Qwen		
	WNS	Cells	Power	WNS	Cells	Power	WNS	Cells	Power	WNS	Cells	Power	WNS	Cells	Power	WNS	Cells	Power	WNS	Cells	Power	WNS	Cells	Power
accu	+0.00	+893	1.37	-0.04	+1.17k	1.68	-0.01	+901	1.36	-0.05	+903	1.38	-0.03	+967	1.41	-0.17	+885	1.40	+0.00	+1.00k	1.46	-0.04	+827	1.30
add_sub	-0.12	+353	0.38	-0.07	+599	0.37	-0.08	+592	0.38	-0.12	+353	0.38	-0.06	+367	0.45	-0.10	+611	0.37	-0.06	+367	0.45	-0.07	+323	0.37
adder_32bit	-0.21	+2.76k	0.64	-0.23	+2.65k	0.64	-0.14	+1.71k	0.49	-0.07	+1.53k	0.44	-0.07	+1.53k	0.44	-0.07	+1.53k	0.44	-0.11	+1.73k	0.49	-0.07	+1.53k	0.44
adder_pipe_64bit	-0.23	+11.8k	7.93	+0.00	+12.7k	8.09	-0.23	+11.8k	7.93	-0.23	+11.8k	7.93	-0.23	+11.8k	7.93	-0.33	+12.1k	7.76	-0.22	+12.2k	7.79	-0.23	+11.8k	7.93
adder_select	+0.00	+1.29k	0.50	+0.00	+1.29k	0.50	+0.00	+1.29k	0.50	+0.00	+1.58k	0.60	+0.00	+1.29k	0.50	+0.00	+1.29k	0.50	+0.00	+1.29k	0.50	+0.00	+1.29k	0.50
addr_calcu	-0.05	+1.14k	0.55	-0.07	+888	0.45	-0.09	+1.37k	0.51	-0.04	+813	0.37	-0.02	+928	0.59	-0.23	+839	0.64	-0.09	+1.35k	0.44	-0.02	+928	0.59
addr_calcu_pipe	+0.00	+1.19k	0.70	-0.05	+1.09k	0.69	-0.08	+2.14k	0.94	-0.08	+2.04k	0.73	-0.10	+1.61k	0.65	+0.00	+2.34k	1.13	-0.02	+2.35k	1.04	+0.00	+1.90k	1.23
alu	-1.05	+5.62k	0.47	-0.99	+4.76k	0.44	+0.00	+5.15k	0.56	-0.29	+5.58k	0.47	+0.00	+5.16k	0.60	-0.29	+5.58k	0.47	+0.00	+5.19k	0.59	-1.37	+6.08k	0.68
alu_64bit	+0.00	+3.69k	0.68	-0.49	+5.50k	0.83	+0.00	+4.77k	0.71	+0.00	+3.89k	0.59	+0.00	+3.43k	0.65	+0.00	+5.01k	0.72	-0.13	+5.13k	0.81	+0.00	+4.86k	0.71
calculation	+0.00	+1.91k	0.65	+0.00	+2.02k	0.55	+0.00	+2.26k	0.63	+0.00	+2.14k	0.66	+0.00	+2.09k	0.66	+0.00	+2.02k	0.55	+0.00	+2.02k	0.64	+0.00	+2.03k	0.61
calculation_pipe	+0.00	+3.39k	1.38	+0.00	+3.21k	0.83	+0.00	+3.32k	0.97	+0.00	+2.77k	0.95	+0.00	+3.35k	0.93	+0.00	+3.21k	0.88	+0.00	+3.22k	0.81	+0.00	+3.57k	1.08
divider_16bit	+0.00	+7.03k	11.2	-0.18	+10.3k	24.6	-0.03	+10.3k	25.0	-0.18	+10.3k	24.6	+0.00	+10.3k	24.5	-0.18	+10.3k	24.6	-0.18	+10.3k	24.6	-0.18	+10.3k	24.6
divider_32bit	+0.00	+27.4k	94.5	+0.00	+35.5k	12.4	+0.00	+68.9k	306	+0.00	+35.6k	12.1	+0.00	+30.3k	209	+0.00	+27.8k	57.1	+0.00	+26.5k	53.0	-0.22	+49.6k	213
divider_32bit_pipe	+0.00	+44.2k	8.23	+0.00	+57.1k	63.3	-1.40	+56.6k	71.1	-1.78	+57.8k	77.0	-1.40	+56.6k	71.1	-2.67	+52.3k	296	-1.40	+56.6k	71.1	+0.00	+57.0k	35.0
fixed_point_adder	+0.00	+1.52k	0.63	-0.20	+1.52k	0.64	-0.22	+1.55k	0.61	+0.00	+1.52k	0.61	-0.51	+1.84k	0.66	-0.29	+1.54k	0.60	-0.37	+1.55k	0.61	-0.22	+1.55k	0.61
float_multi	-0.22	+17.1k	7.87	+0.00	+17.4k	7.79	+0.00	+17.5k	7.96	-0.32	+17.5k	8.46	-0.01	+17.7k	8.20	-0.39	+17.5k	7.84	-0.78	+18.2k	9.00	-0.32	+17.5k	8.46
float_multi_pipe	+0.00	+19.6k	10.1	+0.00	+20.8k	9.12	+0.00	+17.7k	8.80	-0.02	+17.6k	7.58	-0.88	+18.1k	8.69	+0.00	+18.5k	7.91	+0.00	+20.4k	9.42	-0.31	+17.6k	7.35
fsm	-0.15	+552	0.36	-0.18	+606	0.37	-0.13	+591	0.37	-0.15	+552	0.36	-0.04	+593	0.37	-0.71	+873	0.90	-0.11	+549	0.37	-0.15	+552	0.36
fsm_encode	+0.00	+1.14k	1.33	-0.10	+1.73k	2.05	-0.19	+1.88k	2.17	-0.08	+2.01k	2.41	+0.00	+1.13k	0.92	-0.44	+1.73k	2.35	-0.10	+1.85k	0.78	+0.00	+721	1.08
gray	+0.00	+425	0.92	+0.00	+542	0.43	+0.00	+532	0.45	+0.00	+434	0.74	+0.00	+535	0.43	-0.23	+419	0.85	+0.00	+536	0.45	+0.00	+486	0.77
mac	-0.23	+5.78k	2.58	-	-	-	-0.20	+5.77k	2.42	-0.20	+5.77k	2.42	-0.30	+5.65k	2.43	-0.18	+5.19k	3.19	-0.23	+5.78k	2.38	-0.18	+5.19k	3.19
mac_pipe	-0.13	+5.52k	2.32	-0.24	+5.53k	2.51	-0.13	+5.52k	2.52	-0.13	+5.52k	2.52	-0.13	+5.52k	2.52	-0.08	+5.25k	2.64	-0.13	+5.52k	2.52	-0.13	+5.52k	2.52
mul_subexpression	-0.28	+1.17k	0.52	-0.27	+1.17k	0.68	-0.27	+1.16k	0.60	+0.00	+1.19k	0.57	-0.27	+1.17k	0.68	-0.27	+1.17k	0.68	-0.27	+1.17k	0.68	-0.27	+1.17k	0.68
multi_16bit	+0.00	+1.76k	1.73	-0.14	+2.07k	2.08	+0.00	+1.88k	1.61	+0.00	+1.78k	1.68	+0.00	+1.79k	1.63	-0.35	+1.99k	1.93	-0.34	+2.14k	1.58	-0.42	+2.06k	1.96
multi_pipe_4bit	+0.00	+361	0.95	+0.00	+318	0.95	-0.15	+347	1.06	+0.00	+319	0.96	+0.00	+330	0.95	+0.00	+305	1.00	+0.00	+321	0.95	-0.15	+347	1.06
multi_pipe_8bit	+0.00	+2.01k	2.06	-0.34	+1.88k	2.15	-0.34	+1.88k	2.15	-0.34	+1.88k	2.15	-0.37	+1.87k	2.20	-1.73	+1.81k	2.15	-0.79	+2.02k	2.14	-0.51	+1.87k	2.12
radix2_div	-0.15	+2.49k	0.87	-0.12	+2.39k	0.87	-0.12	+2.39k	0.88	-0.12	+2.38k	0.89	-0.19	+2.38k	0.86	-0.19	+2.38k	0.86	-0.19	+2.38k	0.86	-0.19	+2.38k	0.86
serial2parallel	-0.05	+807	1.11	+0.00	+679	1.05	+0.00	+727	1.12	+0.00	+719	1.06	+0.00	+727	1.12	+0.00	+679	0.98	+0.00	+740	1.32	-0.01	+677	1.05
traffic_light	-0.06	+975	0.54	+0.00	+822	0.55	+0.00	+680	1.22	+0.00	+665	0.49	+0.00	+986	0.59	-0.04	+681	1.37	+0.00	+229	0.30	-0.23	+738	0.51
width_8to16	-0.22	+921	2.85	-0.24	+981	2.25	-0.42	+851	2.46	-0.26	+817	2.58	-0.01	+1.94k	4.53	-0.26	+817	2.58	-0.21	+873	1.79	-0.26	+817	2.58
Dom./Dmd.	18/3			5/4			3/2			17/3			5/3			14/7			3/4			5/3		

IV. EXPERIMENTS

A. Experimental Setup

We evaluate coding agents driven by 8 frontier LLMs, including GPT-5.4 [21], Claude Sonnet 4.6 [22], Kimi K2.6 [23], DeepSeek V4 Pro and Flash [24], MiniMax M2.7 [25], GLM 5.1 FP8 [26], and Qwen 3.5 122B [27], on the 30 TicTacBench tasks using the evaluation setup in Section III. All reported closure, WNS, area, power, ADP, and EDDP values are computed from the post-PnR evaluation. The evaluation flow is fully automated, reproducible, and open-sourced.

B. Overall Closure and QoR Results

Figure 3 gives the main leaderboard. The strongest agent by Timing Closure Rate is the GPT-5.4-based agent, which closes 53.3% of tasks, while the DeepSeek V4 Pro-based agent has the largest Avg. Δ WNS%. The next group includes the Kimi K2.6-, Claude Sonnet 4.6-, and DeepSeek V4 Pro-based agents, with Timing Closure Rates around 40%. The remaining agents close fewer tasks, and the MiniMax M2.7-based agent is the only one whose Avg. Δ WNS% is slightly negative. These results show that even state-of-the-art agents close only about half of TicTacBench at best; 23% of tasks are not closed by any agent, and the optimizations from all agents almost always incur substantial area degradation.

The gap between Timing Closure Rate and Avg. Δ WNS% shows why both timing metrics are needed. The GPT-5.4-based agent has the highest Timing Closure Rate, but its Avg. Δ WNS% (48.7%) is lower than that of the DeepSeek V4 Pro-based agent (55.9%) and the Kimi K2.6-based agent (53.0%). This result is consistent with the GPT-5.4-based agent making larger RTL rewrites: such rewrites can remove the original bottleneck and close timing, but they can also introduce new

critical paths on some tasks. The DeepSeek V4 Pro-based agent shows a different pattern. It obtains the best Avg. Δ WNS% but closes only 40.0% of tasks, suggesting that local repairs often improve slack but do not always change the design enough to meet the clock constraint.

The EDDP and ADP metrics further show the PPA cost of timing closure. Most agents improve EDDP, led by the GLM 5.1 FP8-based agent at 13.4%, but every agent degrades ADP, from -2.5% for the GPT-5.4-based agent to -11.9% for the DeepSeek V4 Pro-based agent. This indicates that current agents primarily obtain timing improvement by spending area. By contrast, reducing delay through power-related mechanisms is harder, because it often depends on lower switching activity, clock-tree optimization, or voltage-frequency choices rather than only local RTL rewriting.

Table II reports the PPA metrics, i.e., WNS, cell count, and power of each agent on each design task. The table also counts whether each optimized result Pareto-dominates, or is dominated by, the corresponding suboptimal result across all three PPA metrics. The GPT-5.4-based agent has the most dominating outcomes (18/30), followed by the DeepSeek V4 Flash-based agent (17/30), while the MiniMax M2.7-based agent has the most dominated outcomes (7/30). These counts show that LLM-driven agents are not consistently Pareto-improving optimizers: stronger agents can find favorable timing-area-power trade-offs more often, but aggressive timing repairs can also produce PPA regressions. Thus, current agents show partial ability to navigate PPA trade-offs, but they still lack stable control over when a timing improvement justifies the associated area or power cost.

```

reg [15:0] sum [3:0];
reg [15:0] mul_out_reg;
always @(posedge clk or negedge rst_n)
  if (!rst_n) mul_out_reg <= 'd0;
  else mul_out_reg <= sum[0] + sum[1] + sum[2] + sum[3];

```



```

reg [11:0] pair_sum0_reg;
reg [15:0] pair_sum1_reg;
wire [15:0] final_sum;
assign final_sum = {4'b0, pair_sum0_reg} + pair_sum1_reg;
always @(posedge clk or negedge rst_n)
  if (!rst_n) pair_sum0_reg <= 'd0;
  else pair_sum0_reg <= {2'b0, sum0} + sum1;

```



Fig. 4. Representative case from multi_pipe_8bit. The failed rewrite leaves a wide addition cone on the critical stage. The successful rewrite moves the register boundary into the reduction tree.

C. Case Study

Figure 4 shows a representative case from multi_pipe_8bit, where the GPT-5.4-based agent is the only agent that closes post-PnR timing. The failed solution keeps the original final reduction shape: four 16-bit partial sums are added in one registered stage before driving the output. This leaves a wide four-input addition cone on the critical stage, and the worst failed run has -1.73 ns post-PnR WNS with 35 setup violations.

In contrast, the successful run performs a retiming instead of simply appending a pipeline stage. It replaces the original output-side reduction register with two pair-sum registers, then computes the final sum from those registered pair sums. The transformation shortens the critical stage from a four-input 16-bit reduction to narrower pair additions followed by one final add, which is the structural change needed for timing closure.

Findings of RQ1: We evaluated timing closure with metrics of Timing Closure Rate, WNS improvement, EDDP improvement, and ADP improvement. Even the best agent can close only 53.3% of tasks. Successful optimizations perform micro-architectural changes, such as retiming, pipeline-boundary movement, and narrower arithmetic/reduction cones.

V. CAPABILITIES ANALYSIS

This section analyzes the agent trajectories produced during ATC tasks to characterize agent capability beyond final metrics. We first introduce the trajectory semantic analysis method used in this study, then aggregate its results to measure capability boundaries and failure distributions, leading to three findings about current agents' ATC capability.

A. Trajectory Semantic Analysis

Trajectory semantic analysis treats an agent execution trajectory as behavioral evidence: the trajectory records how the agent reasons, acts through tools, observes feedback, and updates its subsequent actions. It is especially useful for analyzing the ATC task process because an agent must read STA evidence, map the path back to RTL, choose a structural rewrite, validate equivalence, and decide whether the candidate has enough margin for physical implementation. The final post-PnR score observes only the outcome of this process.

TABLE III
CAPABILITY AND FAILURE CATEGORIES OF TRAJECTORY SEMANTIC ANALYSIS.

Capability	Failure category	Meaning
Domain Knowledge	Inefficient STA report-driven optimization	Fails to map key STA information such as slack and critical paths to the RTL level for report-driven optimization.
	Suboptimal datapath design	Preserves excessively deep arithmetic/datapath logic in the design.
Architecture Design	Suboptimal pipeline design	Fails to partition pipeline stages to cut the critical combinational logic chain, or introduces a new one.
	Suboptimal data/control-path design	Introduces control-path logic such as decode or mux logic on critical datapath paths.
Optimization Flow	Unsound exploration	Explores only locally suboptimal solutions and lacks a plan for exploring globally optimal solutions.
	Unsound acceptance	Prematurely accepts weakly converged results from pre-PnR STA, leading to failure at post-PnR.

To apply trajectory semantic analysis, we export and preserve 240 complete agent trajectories. Each trajectory contains the task input and final output, tool calls and tool results, and the agent's reasoning traces during the run. We apply trajectory semantic analysis using a lightweight analyst LLM, specifically DeepSeek V4 Flash, to analyze the trajectories. We further compare runs horizontally across agents on the same task. For each failed run, we pair it with the same-task successful run that has the strongest post-PnR WNS. We then identify the semantic differences between trajectories that converge successfully and those that fail to converge, cluster the differences, and obtain the 6 major failure categories.

B. Failure Mechanisms and Findings

Table III summarizes the taxonomy from the trajectory semantic analysis. The 6 failure categories are grouped into three capabilities: RTL domain knowledge, RTL architecture design, and optimization flow.

Finding 1: report-driven optimization strongly affects timing-closure outcomes. Figure 5 shows that lower-closure agents generally fail more often on STA report-driven optimization, while higher-closure agents fail less on this dimension. This pattern suggests that the ability to turn STA reports into targeted RTL changes has a strong effect on final optimization success. The failed trajectories often begin by reading substantial timing-report content, but the subsequent RTL edits are not derived from the reported critical paths, cells, or slack trends. Instead, the agent falls back to common RTL optimization templates and repeats ineffective cycles of RTL editing and STA analysis. Successful trajectories, by contrast, convert report evidence into targeted RTL changes. For example, in accu, the successful GPT-5.4-based agent run mapped the `_148_/Q` → `_155_/D` endpoint path to the accumulator carry/mux structure and rewrote that structure. Failed peers made plausible state or adder edits, but those edits did not shorten the same post-PnR path.

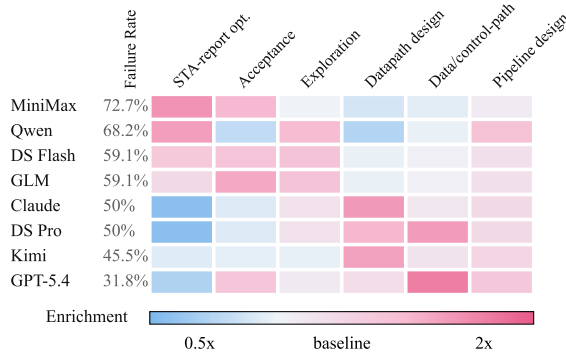


Fig. 5. Failure mechanism profiles by failed trajectories. The heatmap reports each agent’s failure-mechanism enrichment relative to the all-agent baseline.

Finding 2: agents often fail to move from local RTL edits to architectural optimization. Many failed trajectories make syntactically valid and functionally equivalent RTL changes, but the edits remain close to the original design and do not change the microarchitecture seen by post-PnR STA. This appears in 46.9% of the failures, where the failed run misses at least one architectural RTL change involving datapath topology, pipeline partitioning, or control/decode placement. Successful trajectories usually make a larger rewrite: ripple or carry-chain logic becomes a Kogge-Stone prefix adder, variable-shift logic becomes sequential shift-and-add, a wide compare or subtract is narrowed, zero flags are computed inside case branches before the result mux, or intermediate pipeline registers split a deep accumulation stage. Failed trajectories instead stay with local optimizations, such as rewriting the + operator as separate assignments, trying similar adder variants, changing case/assign style, or adding a register that does not cut the actual critical path. In `adder_pipe_64bit`, failed runs repeatedly explored similar adder variants and accepted the built-in plus operator as locally best under pre-PnR checks. The successful run instead pre-registered generate/propagate values and used a Kogge-Stone prefix structure. These examples show that repeated STA runs are not enough unless the agent can leave a local edit pattern and choose an architecturally stronger RTL design.

Finding 3: different agents fail at different stages of timing closure. Figure 5 shows that the failure categories identify which capability is missing in each agent. For lower-closure agents, the main bottleneck is still the basic optimization loop. In the MiniMax M2.7, Qwen 3.5 122B, DeepSeek V4 Flash, and GLM 5.1 FP8 based agents, 76.9%–81.25% of failures fall into Inefficient STA report-driven optimization, Unsound exploration, or Unsound acceptance. These runs may read reports and produce valid RTL, but they do not reliably choose the right RTL change, compare enough alternatives, or reject a candidate that is too weak for post-PnR closure. For higher-closure agents, the remaining bottleneck is more often the RTL design choice itself: in the Claude Sonnet 4.6, DeepSeek V4 Pro, GPT-5.4, and Kimi K2.6 based agents, 71.4%–81.8% of failures fall into Suboptimal datapath design, Suboptimal

pipeline design, or Suboptimal data/control-path design. The correlations support the same view: Timing Closure Rate is negatively correlated with Inefficient STA report-driven optimization (-0.70) and Optimization Flow failures (-0.41), but positively correlated with Suboptimal datapath design (+0.81).

Findings of RQ2: The 6 failure categories expose missing capabilities of coding agents in RTL domain knowledge, RTL architecture design, and optimization flow. Lower-closure agents mainly fail in report-driven optimization and optimization-flow decisions, while higher-closure agents’ remaining failures concentrate in RTL design trade-offs.

VI. TICTACSKILL: AGENT SKILL FOR TIMING CLOSURE

In this section, we introduce **TicTacSkill**, a plug-and-play method designed to improve coding agents on ATC tasks. We first describe how **TicTacSkill** derives SOPs for timing closure from the failure analysis identified in Section V. We then evaluate this method and show its improvement over the original agent baseline.

A. Skill Design

Based on the analysis in Section V, we propose **TicTacSkill**, an agent skill that encodes the failure categories identified in Section V as Standard Operating Procedures (SOPs) for ATC. Before solving a task, the coding agent receives this skill that specifies how to inspect timing evidence, localize critical paths, propose RTL rewrites, validate candidates, and decide when to pivot.

Critical Path Localization. For Inefficient STA report-driven optimization, the skill requires the agent to map slack, startpoint, endpoint, dominant operators, and suspected RTL assignments to the actual critical cone before editing.

Datapath Design. For Suboptimal datapath design, the skill requires the agent to state which measurable property the candidate changes, such as logic depth, operation width, mux order, fanout, register boundary, or worst-path class.

Pipeline Design. For Suboptimal pipeline design, the skill requires evidence that a new or moved register actually splits the violating stage rather than only adding latency around the same combinational cone.

Control/Decode Placement. For Suboptimal data/control-path design, it checks whether flags, comparisons, enables, and state/decode logic are serialized after wide data movement and therefore should be moved earlier or narrowed.

Evidence-Driven Search. For Unsound exploration, it forces each iteration to compare candidates by correctness, WNS/TNS, violation count, and worst-path evidence, and to abandon repeated edits that do not change the limiter.

Pre-PnR/Post-PnR Mismatch. For Unsound acceptance, it treats nominal pre-PnR closure as insufficient unless the candidate also has margin or a stated calibration to the final post-PnR objective.

B. Experimental Results

We evaluate **TicTacSkill** by comparing each skill-enabled agent run against the corresponding original run under the

TABLE IV

TicTacSkill ABLATION ON THE THREE MODELS WITH SKILL-ENABLED RUNS. EACH W/ SKILL ROW IS PAIRED WITH THE SAME MODEL AND TASK WITHOUT THE SKILL.

Model	Setting	Closure	Avg. Δ WNS%	Avg. Δ EDDP%	Avg. Δ ADP%
GPT-5.4	w/o skill	53.3%	48.7%	5.5%	-2.5%
	w/ skill	56.7%	66.4%	16.3%	-3.6%
MiniMax	w/o skill	23.3%	-1.7%	-0.7%	-9.6%
	w/ skill	36.7%	17.0%	-7.2%	-10.9%
DeepSeek	w/o skill	33.3%	44.6%	11.6%	-4.3%
	w/ skill	43.3%	62.1%	14.9%	-3.0%

same agent and task settings. Table IV reports the ablation results. Across all three agents, TicTacSkill improves Timing Closure Rate and Avg. Δ WNS%, showing that the failure categories can be converted into useful guidance for timing repair. The GPT-5.4-based agent improves from 53.3% to 56.7% Timing Closure Rate and from 48.7% to 66.4% Avg. Δ WNS%. The MiniMax M2.7-based agent shows the largest gain in Timing Closure Rate, increasing from 23.3% to 36.7%, while its Avg. Δ WNS% moves from -1.7% to 17.0%. The DeepSeek V4 Flash-based agent improves from 33.3% to 43.3% Timing Closure Rate and from 44.6% to 62.1% Avg. Δ WNS%.

Findings of RQ3: Targeting the observed failure categories with explicit timing-closure SOPs improves agents' Timing Closure Rate and WNS improvement on the tested agents. This shows that the TicTacSkill pattern of turning trajectory-derived failure analysis into a reusable agent skill is effective for improving agents' timing-closure capability.

VII. DISCUSSION AND LIMITATIONS

TicTacBench is an initial benchmark with 30 timing-closure tasks. This scale is sufficient for exposing clear capability gaps and recurring failure mechanisms, but in future versions we will expand the task set to improve statistical coverage and support finer-grained comparisons across model families. The current task setting is also focused on single-module combinational designs and single-clock sequential designs. This scope keeps the benchmark controllable and reproducible, but it does not yet cover the broader range of RTL timing-closure cases found in larger designs. In future benchmark extensions, we will include more diverse module structures, timing bottlenecks, and design patterns so that ATC evaluation can measure agent capability across a wider task distribution.

VIII. CONCLUSION

This paper introduces Agent-based Timing Closure as an evaluation target for coding agents that must repair RTL using timing evidence and are judged by post-PnR setup closure. Across more than 300 runs of coding agents driven by 8 frontier LLMs, the best agent closes only 53.3% of tasks, showing that current agents still struggle to achieve real timing closure. TicTacBench can serve as a foundation for moving RTL agents from generating correct modules toward solving implementation-constrained hardware optimization tasks.

REFERENCES

- [1] C. E. Jimenez *et al.*, "Swe-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQM66>
- [2] S. Liu *et al.*, "Rtlcoder: Fully open-source and efficient llm-assisted rtl code generation technique," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 4, pp. 1448–1461, 2025.
- [3] W. Lv *et al.*, "Vitat: Timing violation-aware debugging of rtl code using large language models," 2025.
- [4] X. Yao *et al.*, "Rtlrewriter: Methodologies for large models aided rtl code optimization," in *IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 98:1–98:7.
- [5] Y. Wang *et al.*, "Symrtlo: Enhancing rtl code optimization with llms and neuron-inspired symbolic reasoning," 2025.
- [6] N. Pinckney *et al.*, "Revisiting VerilogEval: A Year of Improvements in Large-Language Models for Hardware Code Generation," Feb. 2025.
- [7] S. Liu *et al.*, "Openllm-rtl: Open dataset and benchmark for llm-aided design rtl generation," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2025, pp. 60:1–60:9.
- [8] A. Allam *et al.*, "Rtl-repo: A benchmark for evaluating llms on large-scale rtl design projects," 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/11310982>
- [9] N. Pinckney *et al.*, "Comprehensive verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on rtl design and verification," 2025.
- [10] Z. Fang, R. Chen, Y. Guo, H. Dai, and L. Wang, "Rtlbench: A multi-dimensional benchmark suite for evaluating llm-generated RTL code," in *43rd IEEE International Conference on Computer Design, ICCD 2025, Richardson, TX, USA, November 10-12, 2025*. IEEE, 2025, pp. 566–573.
- [11] M. Liu *et al.*, "VerilogEval: Evaluating large language models for verilog code generation," 2023.
- [12] Y. Lu *et al.*, "A new benchmark for the appropriate evaluation of rtl code optimization," 2026.
- [13] A. B. Kahng *et al.*, *Timing Closure*. Springer Netherlands, 2011, pp. 219–264.
- [14] C. E. Leiserson *et al.*, "Retiming synchronous circuitry," *Algorithmica*, vol. 6, no. 1, pp. 5–35, 1991.
- [15] C. Chen *et al.*, "E-syn: E-graph rewriting with technology-aware cost functions for logic synthesis," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 124:1–124:6.
- [16] V. S. Livramento *et al.*, "Clock-tree-aware incremental timing-driven placement," *ACM Transactions on Design Automation of Electronic Systems*, 2016.
- [17] W. Jiang *et al.*, "Ir-aware eco timing optimization using reinforcement learning," in *Proceedings of the ACM/IEEE Workshop on Machine Learning for CAD*, 2024, p. 7.
- [18] M. Shalan and T. Edwards, "Building openlane: A 130nm openroad-based tapeout-proven flow: Invited paper," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–6.
- [19] M. R. Stan and K. Skadron, "Power-aware computing," *Computer*, vol. 36, no. 12, pp. 35–38, 2003.
- [20] A. B. Chowdhury *et al.*, "Invictus: Optimizing boolean logic circuit synthesis via synergistic learning and search," 2023.
- [21] OpenAI, "Introducing gpt-5.4," 2026, published: 2026-03-05, Accessed: 2026-03-30. [Online]. Available: <https://openai.com/index/introducing-gpt-5-4/>
- [22] Anthropic, "Introducing claude sonnet 4.6," 2026, published: 2026-02-17, Accessed: 2026-03-30. [Online]. Available: <https://www.anthropic.com/news/claude-sonnet-4-6>
- [23] Moonshot AI, "Kimi k2: Open agentic intelligence," 2025.
- [24] DeepSeek-AI, "Deepseek-v4: Towards highly efficient million-token context intelligence," DeepSeek-AI, Tech. Rep., 2026, technical report.
- [25] MiniMax, "The minimax-m2 series: Mini activations unleashing max real-world intelligence," 2026.
- [26] GLM-5-Team *et al.*, "Glm-5: from vibe coding to agentic engineering," 2026.
- [27] A. Yang *et al.*, "Qwen3 technical report," 2025.